

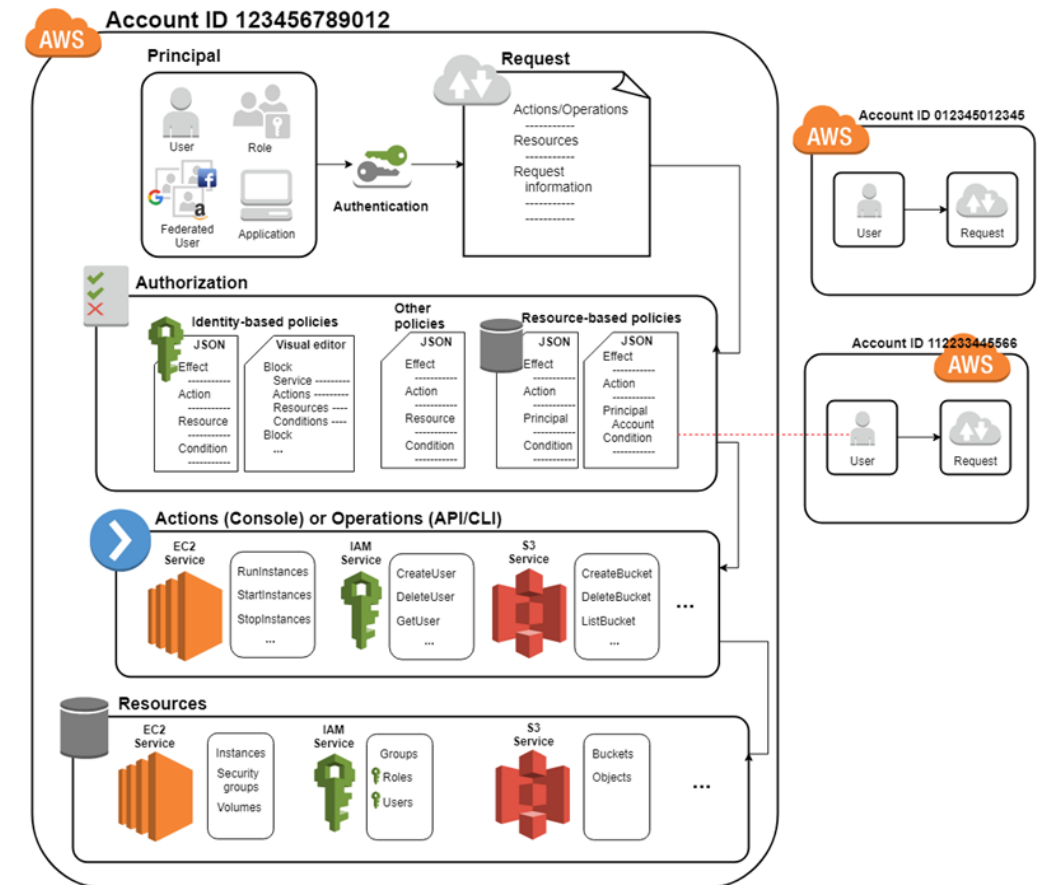
Supply Chain Security, Information Systems Acquisition, Development, and Maintenance

Dr. Ruba Alomari

MITS5600G – Security Policies and Risk Management

Recall – Last Week – Identity and Access Management (IAM)

- The process for identifying, authenticating, and authorizing users or groups of users to have access to applications, systems, or networks is referred to as identity management.
- It is done by **associating user permissions with established identities.**
- Identity management is focused on authentication, whereas access management is aimed at authorization.



How IAM works

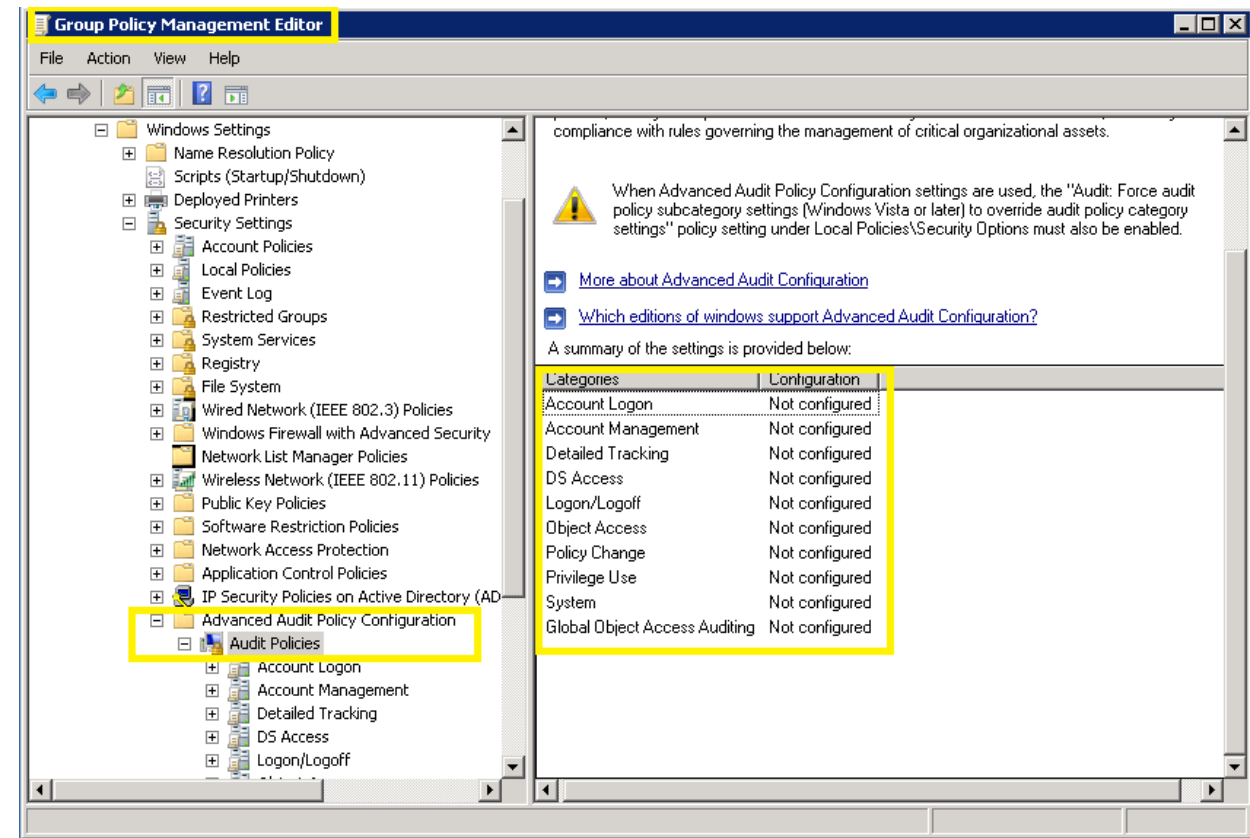
<https://docs.aws.amazon.com/IAM/latest/UserGuide/intro-structure.html>

Recall – Last Week – Categories of ACLs

- Main models of access control:
 - Mandatory Access Control (MAC)
 - Discretionary Access Control (DAC)
 - Role-based Access Control (RBAC)
 - Attribute-based access control (ABAC)
 - Rule-based access control

Recall – Last Week – Types of Access to be Monitored

- **Successful** access: a record of user activity.
- **Failed** access: indicative of either unauthorized attempts or authorized user issues.
- **Privileged** operations: compromise or misuse of administrator accounts can have disastrous consequences.



Topics

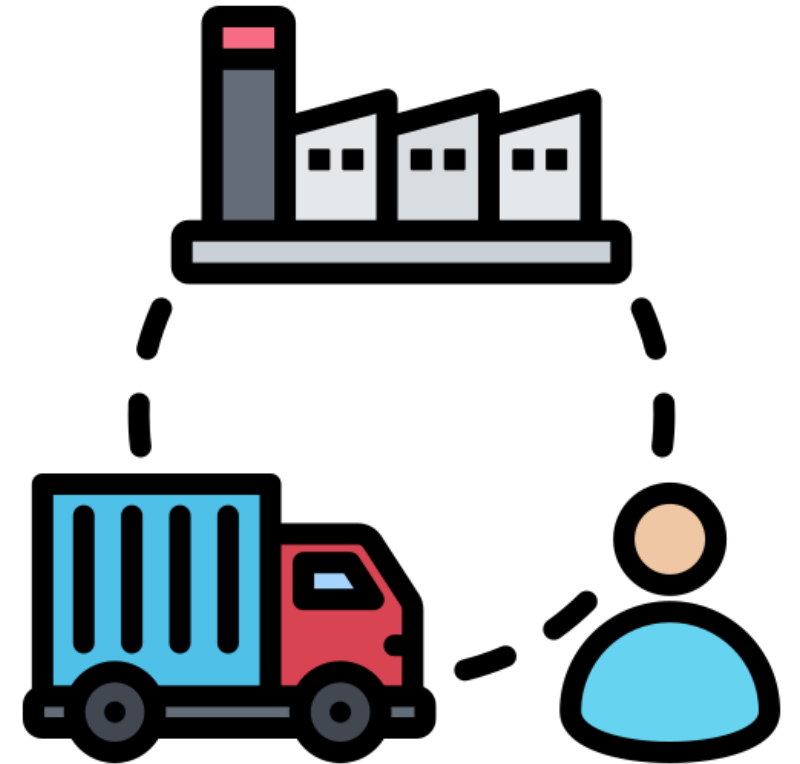
- Supply Chain Security
- System Security Requirements
- Secure Code
- Cryptography

Topics

- Supply Chain Security
- System Security Requirements
- Secure Code
- Cryptography

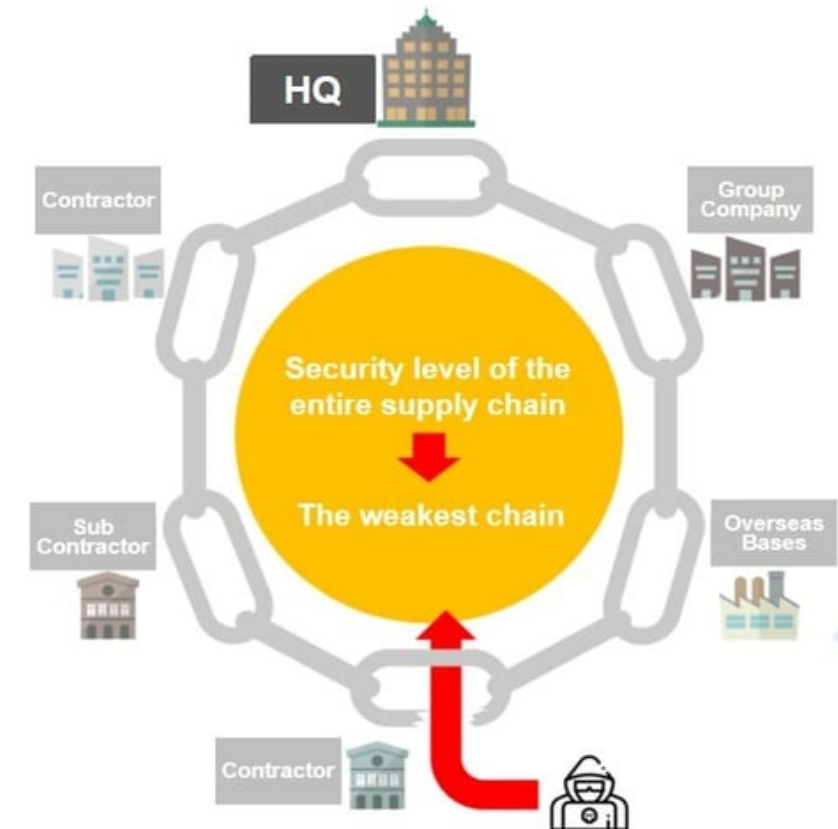
Why is Supply Chain Security Important?

- Supply chain security is critical for several reasons:
 1. **Protection of Sensitive Information:** Prevents compromise of sensitive information such as customer personal information and proprietary product data.
 2. **Integrity of Goods:** Ensures the integrity of goods being delivered. Especially in industries like pharmaceuticals, where counterfeit or tampered with products can pose significant health risks.
 3. **Business Reputation:** Maintains trust among customers and partners through reliable and secure supply chain practices.



Emerging Threats to Supply Chains

- The complexity of modern supply chains presents numerous security challenges.
 1. **Cyber Attacks:** Target weak links (small suppliers) to breach larger networks, risking significant data loss.
 2. **Physical Threats:** Include theft and terrorism, especially dangerous for high-value goods during transit.
 3. **Global Vulnerabilities:** Geopolitical tensions and natural disasters can disrupt global supply chains, impacting production and distribution 📄.



<https://www.nri-secure.com/>

Strategies for Enhancing Supply Chain Security

- Strategies for Enhancing Supply Chain Security:
 1. Risk Assessment and Management
 2. Technology Utilization
 - Blockchain
 - Encryption and IDPS
 3. Stakeholder Collaboration

1. Risk Assessment and Management

- **Risk Assessment:** Identify potential vulnerabilities within the supply chain:
 - from raw materials sourcing to product delivery;
 - regularly evaluate supplier and partner security practices.
- **Risk Management:** Develop a strategy incorporating periodic reviews and continuous updates to adapt to evolving threats.

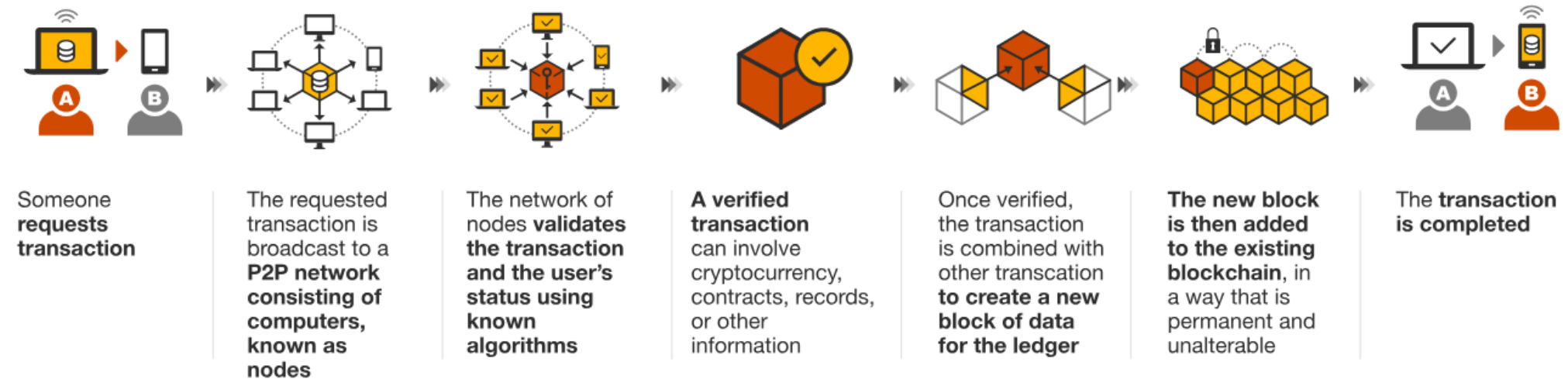


<https://supplychain-risk.com/>

2. Technology Utilization

- **Blockchain:** Improves transparency and traceability, helps verify authenticity, and detects product tampering.

How blockchain works



<https://www.pwc.com/>

How do Different Industries use Blockchain?

1. Energy:

- **Peer-to-peer energy trading:** Smart contracts and smart meters enable automatic energy exchanges between producers (e.g., homeowners with solar panels) and consumers.

2. Finance:

- **Banks and financial institutions** use blockchain for managing online payments, accounts, and trading, significantly improving efficiency and reducing manual processes (e.g., Singapore Exchange Limited).

3. Media and Entertainment:

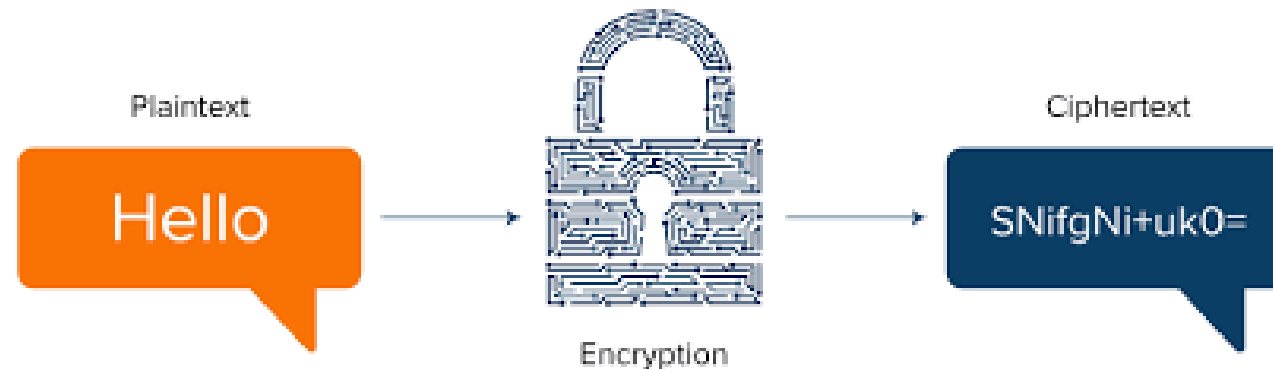
- **Digital rights management:** Companies use blockchain systems to manage copyright data (e.g., [Sony Music Entertainment Japan](#)).

4. Retail:

- **Supply chain tracking:** Retailers (e.g., Amazon) use blockchain to verify product authenticity, track goods from suppliers to consumers, and maintain a transparent ledger accessible to all stakeholders.

2. Technology Utilization

- **Cybersecurity Measures:** Implement encryption and intrusion detection systems to defend against cyber threats and data breaches.



3. Stakeholder Collaboration

- Given the interconnected nature of supply chains, collaboration among all stakeholders is vital.
- **Sharing information** about threats and best practices can help preempt attacks and mitigate risks.
- Industry consortia and public–private partnerships can facilitate such collaboration, providing **a platform for collective defense** against common threats.



SBOMs Enhance Supply Chain Security /1

- A Software Bill of Materials (SBOMs) is a detailed inventory listing all software components, libraries, modules, their versions, and dependencies.
- SBOM enables an organization to quickly identify and respond to vulnerabilities within its software supply chain.

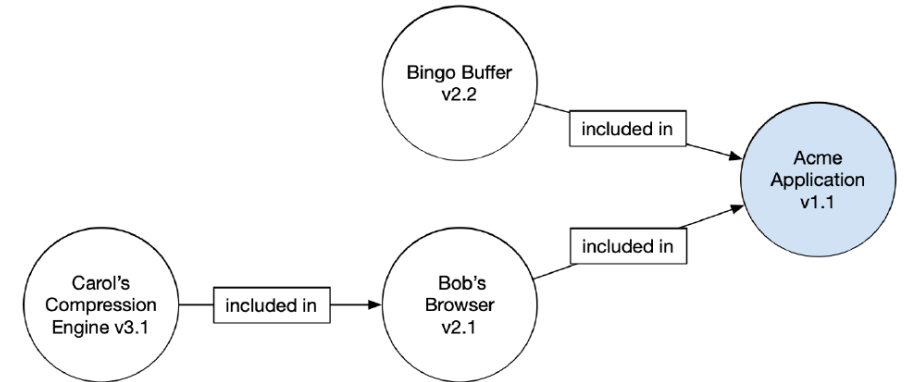


Figure 1: Conceptual SBOM graph

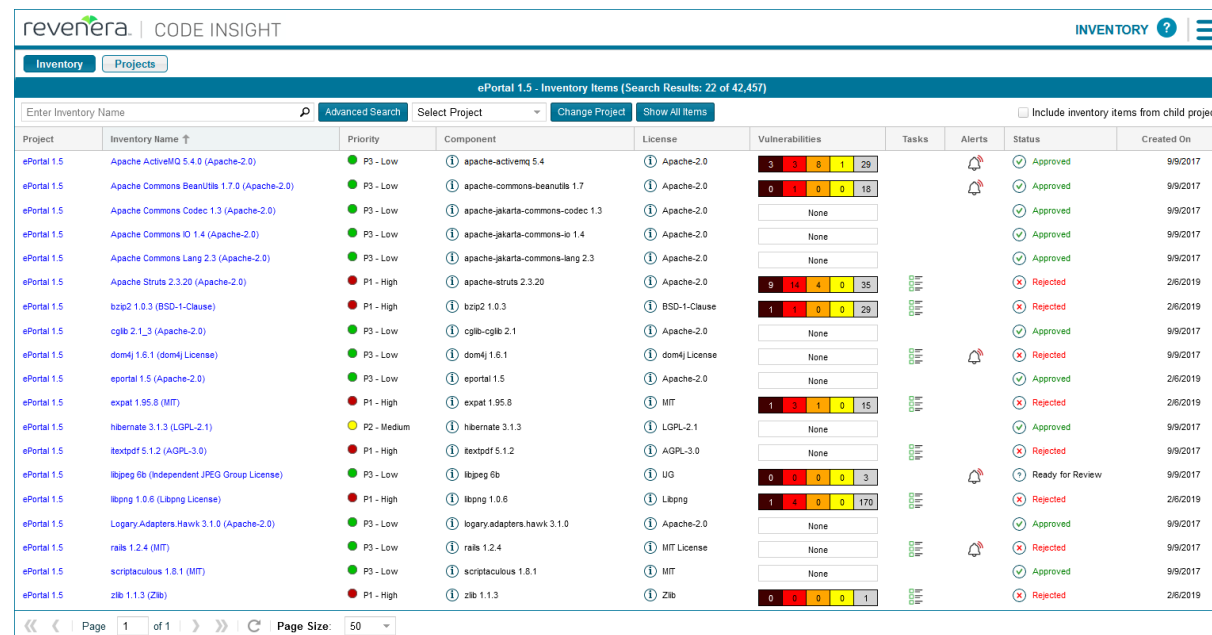
Component Name	Supplier	Version	Author	Hash	UID	Relationship
Application	Acme	1.1	Acme	0x123	234	Primary
--- Browser	Bob	2.1	Bob	0x223	334	Included in
--- Compression Engine	Carol	3.1	Acme	0x323	434	Included in
--- Buffer	Bingo	2.2	Acme	0x423	534	Included in

Table 2: Conceptual SBOM table²⁴

<https://www.ntia.gov/>

SBOMs Enhance Supply Chain Security /2

- Increasing software complexity and widespread use of open-source components require **clear visibility into software composition**.
- When a new vulnerability is discovered in a component listed in an SBOM, an organization can **immediately assess its exposure** and prioritize remediation efforts.



The screenshot displays the 'ePortal 1.5 - Inventory Items' page in the Revenera Code Insight application. The page shows a list of 22 software components with their associated vulnerabilities. The table includes columns for Project, Inventory Name, Priority, Component, License, Vulnerabilities, Tasks, Alerts, Status, and Created On. The vulnerabilities are color-coded: red for high severity, yellow for medium, and green for low. The status column indicates whether the component is 'Approved', 'Rejected', or 'Ready for Review'.

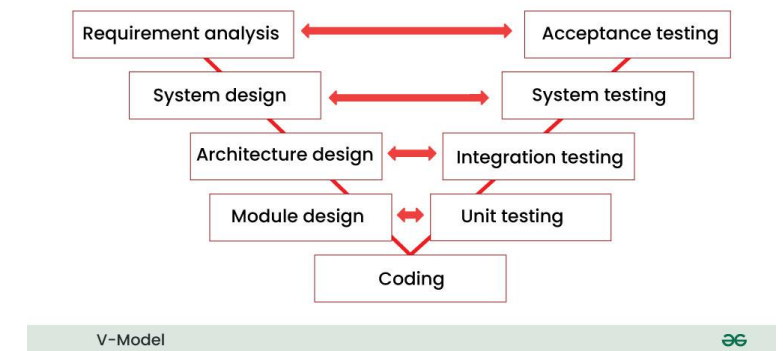
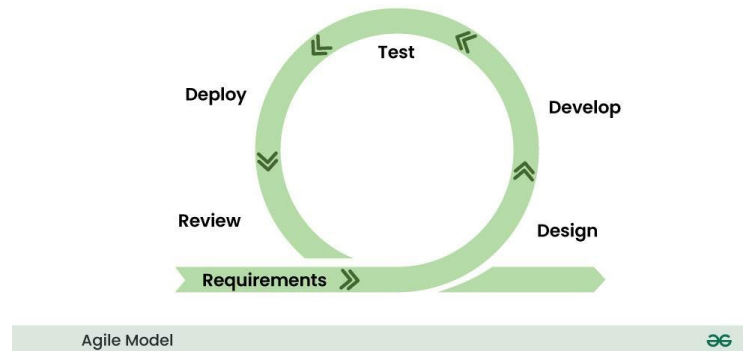
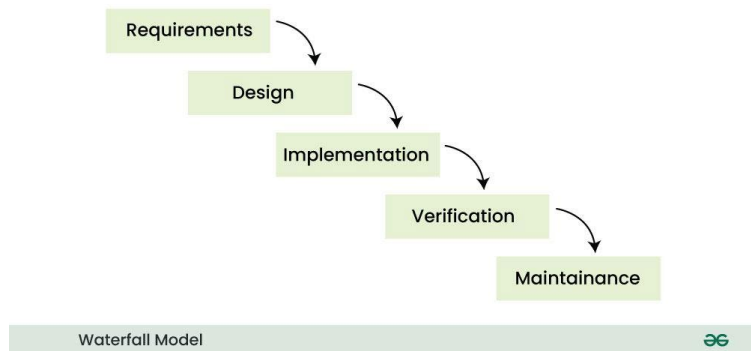
Project	Inventory Name	Priority	Component	License	Vulnerabilities	Tasks	Alerts	Status	Created On
ePortal 1.5	Apache ActiveMQ 5.4.0 (Apache-2.0)	P3 - Low	① apache-activemq 5.4	① Apache-2.0	3 0 0 1 29			Approved	9/9/2017
ePortal 1.5	Apache Commons BeanUtils 1.7.0 (Apache-2.0)	P3 - Low	① apache-commons-beanutils 1.7	① Apache-2.0	0 0 0 0 18			Approved	9/9/2017
ePortal 1.5	Apache Commons Codec 1.3 (Apache-2.0)	P3 - Low	① apache-jakarta-commons-codec 1.3	① Apache-2.0	None			Approved	9/9/2017
ePortal 1.5	Apache Commons IO 1.4 (Apache-2.0)	P3 - Low	① apache-jakarta-commons-io 1.4	① Apache-2.0	None			Approved	9/9/2017
ePortal 1.5	Apache Commons Lang 2.3 (Apache-2.0)	P3 - Low	① apache-jakarta-commons-lang 2.3	① Apache-2.0	None			Approved	9/9/2017
ePortal 1.5	Apache Struts 2.3.20 (Apache-2.0)	P1 - High	① apache-struts 2.3.20	① Apache-2.0	9 14 4 0 35			Rejected	2/6/2019
ePortal 1.5	bsip2 1.0.3 (BSD-1-Clause)	P1 - High	① bsip2 1.0.3	① BSD-1-Clause	1 0 0 0 29			Rejected	2/6/2019
ePortal 1.5	cglib 2.1.3 (Apache-2.0)	P3 - Low	① cglib-cglib 2.1	① Apache-2.0	None			Approved	9/9/2017
ePortal 1.5	dom4j 1.6.1 (dom4j License)	P3 - Low	① dom4j 1.6.1	① dom4j License	None			Rejected	9/9/2017
ePortal 1.5	eportal 1.5 (Apache-2.0)	P3 - Low	① eportal 1.5	① Apache-2.0	None			Approved	2/6/2019
ePortal 1.5	expat 1.95.8 (MIT)	P1 - High	① expat 1.95.8	① MIT	1 0 0 0 15			Rejected	2/6/2019
ePortal 1.5	hibernate 3.1.3 (LGPL-2.1)	P2 - Medium	① hibernate 3.1.3	① LGPL-2.1	None			Approved	9/9/2017
ePortal 1.5	textpdf 5.1.2 (AGPL-3.0)	P1 - High	① textpdf 5.1.2	① AGPL-3.0	None			Rejected	9/9/2017
ePortal 1.5	libjpeg 6b (Independent JPEG Group License)	P3 - Low	① libjpeg 6b	① UG	0 0 0 0 3			Ready for Review	9/9/2017
ePortal 1.5	libpng 1.0.6 (Libpng License)	P1 - High	① libpng 1.0.6	① Libpng	1 0 0 0 170			Rejected	2/6/2019
ePortal 1.5	Log4j-Adapters Hawk 3.1.0 (Apache-2.0)	P3 - Low	① log4j-adapters.hawk 3.1.0	① Apache-2.0	None			Approved	9/9/2017
ePortal 1.5	rails 1.2.4 (MIT)	P3 - Low	① rails 1.2.4	① MIT License	None			Rejected	9/9/2017
ePortal 1.5	scriptaculous 1.8.1 (MIT)	P3 - Low	① scriptaculous 1.8.1	① MIT	None			Approved	9/9/2017
ePortal 1.5	zlib 1.1.3 (Zlib)	P1 - High	① zlib 1.1.3	① Zlib	0 0 0 0 1			Rejected	2/6/2019

Topics

- Supply Chain Security
- System Security Requirements
- Secure Code
- Cryptography

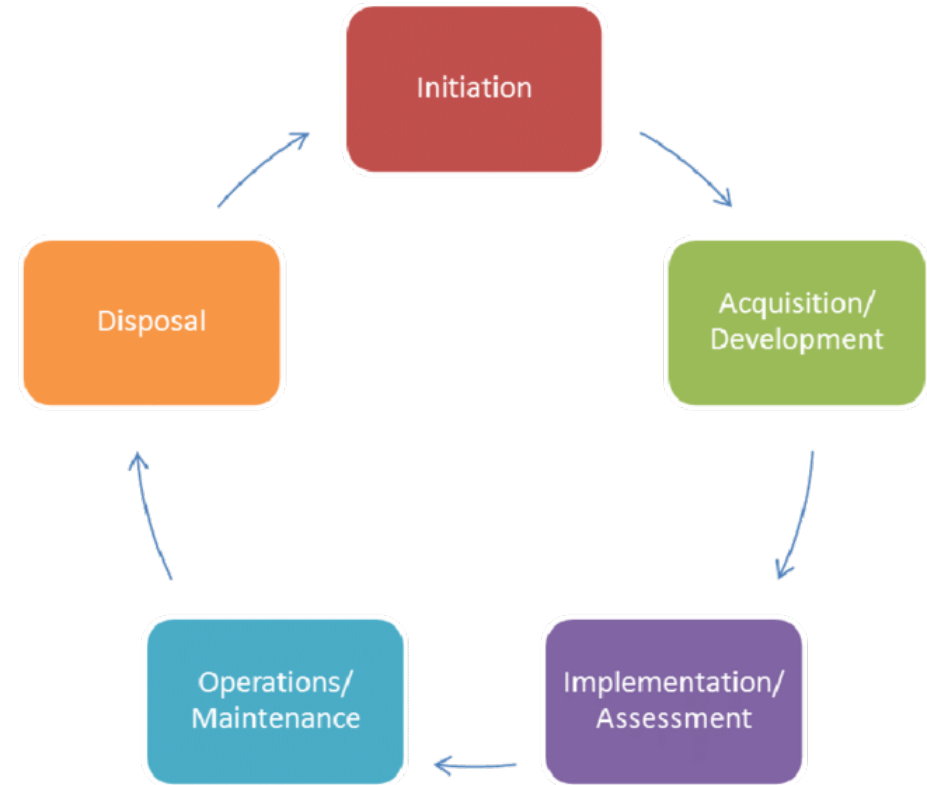
System Security Requirements

- Security should be a priority **during the design and acquisition** phases of any new information system, application, or code development.
- Attempting to retrofit security is expensive, resource-intensive, and often does not work.



What Is SDLC?

- Systems development lifecycle (SDLC) provides a standard process for any system development or acquisition effort.
- As defined by NIST, an SDLC includes five phases:
 1. Initiation
 2. Acquisition/Development
 3. Implementation/Assessment
 4. Operations/Maintenance
 5. Disposal



NIST System Development Life Cycle (SDLC)

SDLC /1

- 1. Initiation Phase:** The need for a system is expressed, and the purpose of the system is documented.
 - Security planning must begin in the initiation phase.
- 2. Development/Acquisition Phase:** The system is designed, purchased, programmed, developed, or otherwise constructed.
 - In addition, the organization should analyze security requirements, perform functional and security testing, and design the security architecture.

Common Application Security (AppSec) Requirements & Techniques



SDLC /2

3. **Implementation/Assessment Phase:** Includes system testing, modification if necessary, retesting if modified, and finally acceptance.
4. **Operations/Maintenance Phase:** The system is put into production.
 - The system is almost always modified by the addition of hardware and software and by numerous other events.
 - Monitoring, auditing, and testing should be ongoing.
 - Configuration management and change control processes are essential to ensure that required security controls are maintained.

Testing Strategies



Static Testing

Identifies software defects without executing the application's code



Structural Testing

Also called White Box Testing, this refers to the software's source code structure to locate defects



Behavioral Testing

Also called Black Box Testing, looks at how the application acts rather than its structural mechanisms

SDLC /3

5. **Disposal Phase:** Often, there is no definitive end or retirement of an information system or code.
- Systems normally evolve or transition to the next generation because of changing requirements or improvements in technology.
 - System security plans should continually evolve with the system.
- Check out
- NIST Special Publication **800-218**: Secure Software Development Framework (SSDF)
<https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-218.pdf>
 - NIST Special Publication **800-218A**: Secure Software Development Practices for Generative AI and Dual-Use Foundation Models: An SSDF Community Profile
<https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-218A.pdf>



In Practice

Systems Development Life Cycle (SDLC) Policy

Synopsis: Ensure a structured and standardized process for all phases of system development/acquisition efforts, which includes security considerations, requirements, and testing.

Policy Statement:

- The Office of Information Technology is responsible for adopting, implementing, and requiring compliance with an SDLC process and workflow. The SDLC must define initiation, development/acquisition, implementation, operations, and disposal requirements.
- At each phase, security requirements must be evaluated and, as appropriate, security controls tested.
- The system owner, in conjunction with the Office of Information Security, is responsible for defining system security requirements.
- The system owner, in conjunction with the Office of Information Security, is responsible for authorizing production systems prior to implementation.
- If necessary, independent experts may be brought in to evaluate the project or any component thereof.

What About Commercially Available or Open Source Software?

- SDLC principles also apply to commercially available software (sometimes referred to as commercial off-the-shelf (COTS) software) and to open source software.
- The primary difference is that the development is not done in-house.

The Register

Malicious xz backdoor reveals fragility of open source

This time, we got lucky. It mostly affected bleeding-edge distros. But that's not a defense strategy

 [Thomas Claburn](#) Mon 1 Apr 2024 // 21:16 UTC

ANALYSIS The discovery last week of a backdoor in a widely used open source compression library called xz could have been a security disaster had it not been caught by luck and atypical curiosity about latency from a Microsoft engineer.

Yet the fortunate find has led industry observers to conclude not much will change to prevent this threat scenario from reoccurring, and that similar, ongoing efforts to compromise software infrastructure may have been missed.

"This might be the best executed supply chain attack we've seen described in the open, and it's a nightmare scenario: malicious, competent, authorized upstream in a widely used library," wrote open source cryptographer Filippo Valsorda. Meanwhile, Michal Zalewski, an ex-Google now veep of security engineering at Snap, described the affair as "one of the most daring infosec capers" ever witnessed.

On Friday, Andres Freund, a principal software engineer at Microsoft and one of the developers of PostgreSQL, posted to the Openwall security mailing list that he had found a backdoor in liblzma, which is part of the xz package.

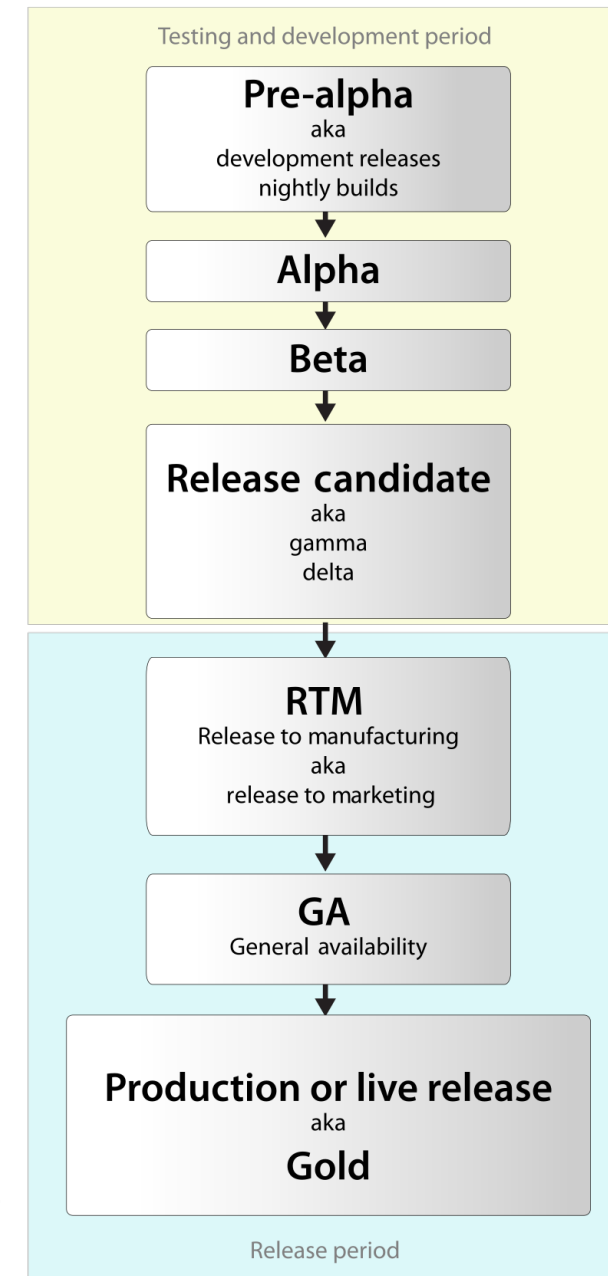
The xz software is used in many Linux distributions and in macOS for tasks like compressing release tarballs, kernel images, and the like.

But the backdoor was caught early – the malicious code only made it into a few bleeding-edge Linux distributions, such as the upcoming Fedora Linux 40; Fedora Rawhide developer distribution; Debian Unstable; and Kali Linux. Vulnerable distributions require glibc (for IFUNC, a way to make indirection function calls into OpenSSH authentication), and xz-5.6.0 or xz-5.6.1

Software Releases /1

- Only **stable** and **tested** software releases should be deployed **on production** servers to protect data availability and data integrity.
- Because software is often released in stages, it is important to be aware of and understand the release stages.

Figure By Heyinsun - Own work, CC BY 3.0,
<https://commons.wikimedia.org/w/index.php?curid=6818861>

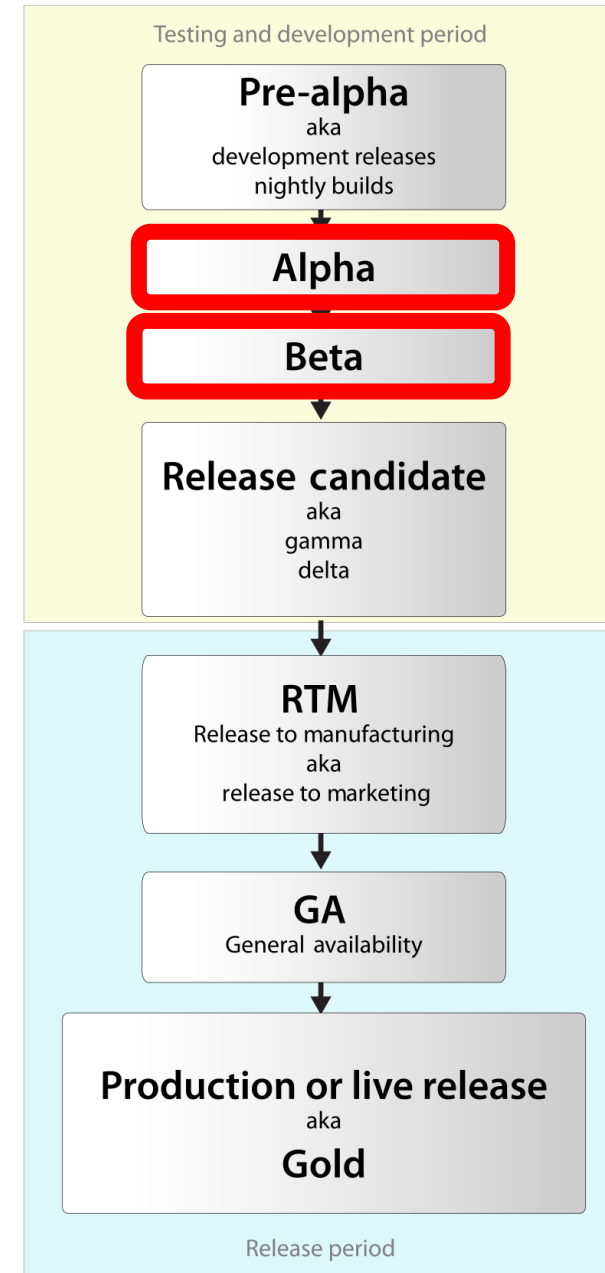


Software Releases /2

- **Alpha phase:**

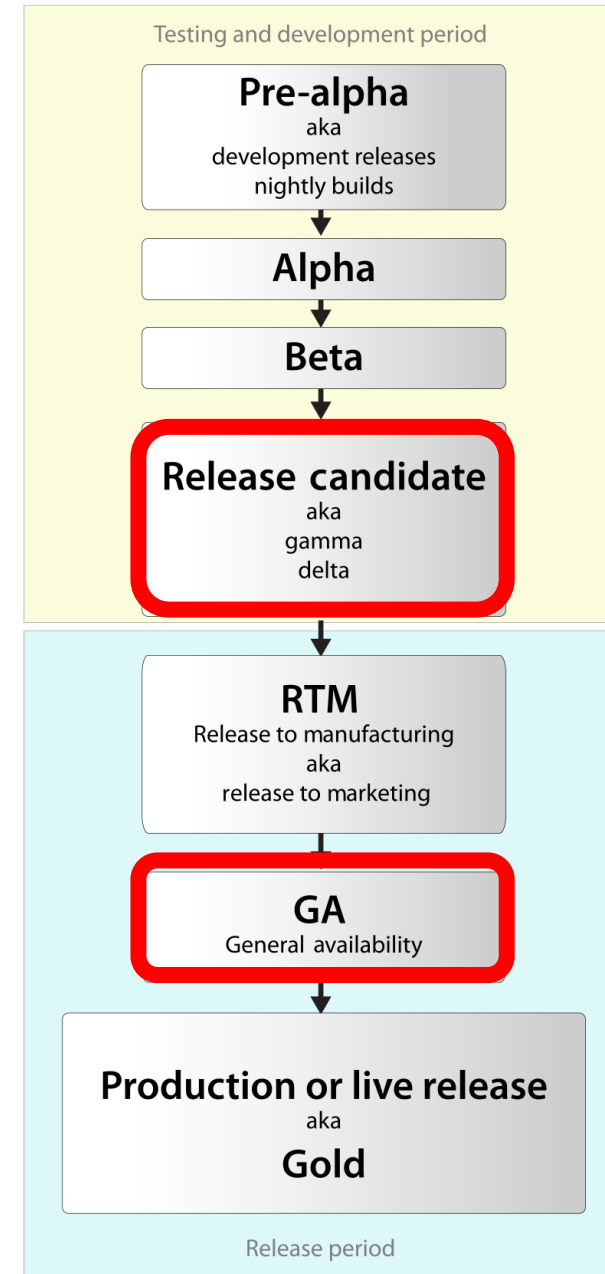
- Initial release of software for testing.
- Can be unstable.
- External availability of alpha software is uncommon in proprietary software.
- Open-source software often has publicly available alpha versions, often distributed as the raw source code of the software.

- **Beta phase:** Software is complete and ready for usability testing.



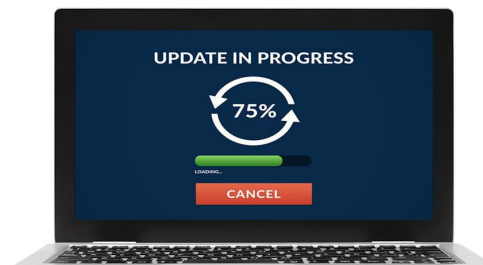
Software Releases /3

- **Release Candidate (RC)**
 - Hybrid of beta and final release version.
 - Has the potential of being final release unless significant issues are identified.
- **General Availability** or go live: Software has been made commercially available.
- Alpha, beta, and RCs have a tendency to be unstable and unpredictable and are not suitable for a production environment.



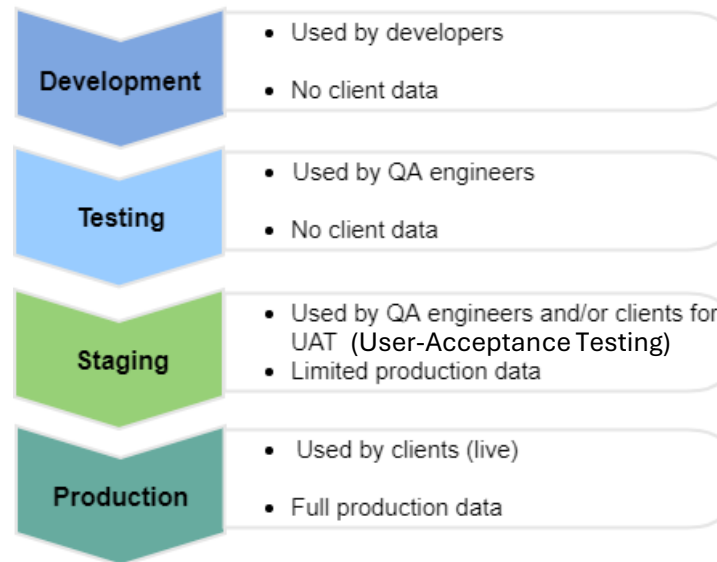
Software Updates

- Updates are different from security patches.
- **Security patches** are designed to address a specific **vulnerability**.
- **Software updates** include **functional** enhancements and new features.
- Updates should be subject to the organization's change management process and should be thoroughly tested.
- A documented rollback strategy should exist before applying any updates.
- If an update requires a system reboot, it should be delayed until the reboot has the least impact on business operations.



The Testing Environment

- Companies should have a test environment.
- The closer to the live environment the test environment is, the more expensive it is, but the more accurate the testing will be.
- The cost of setting up the test environment should be **compared to the cost of a loss of data CIA** due to a patch-related reason.



Protecting Test Data

- Testing environment should be **100% segregated** from the live network.
- Live data should NEVER be used in a test environment.
 - The test servers may not be as well secured as the live production servers.
- **De-identified** or **dummy** data should be used in place of live data.
 - De-identification is the process of removing information that would identify the source or subject of the data.
 - Strategies include deleting or masking the name, social security number, date of birth, and demographics.
 - Dummy data is fictional.

In Practice

System Implementation and Update Policy

Synopsis: Define the requirements for the implementation and maintenance of commercial and open source software.

Policy Statement:

- Operating systems and applications (collectively referred to as “system”) implementation and updates must follow the company’s change management process.
- Without exception, alpha, beta, or prerelease applications must not be deployed on production systems.
- It is the joint responsibility of the Office of Information Security and the Office of Information Technology to test system implementation and updates prior to deployment in the production environment.
- The Office of Information Technology is responsible for budgeting for and maintaining a test environment that is representative of the production environment.
- Without exception, data classified as “protected” must not be used in a test environment unless it has been de-identified. It is the responsibility of the Office of Information Security to approve the de-identification schema.

Topics

- Supply Chain Security
- System Security Requirements
- Secure Code
- Cryptography

Insecure vs. Secure Code /1

- **Insecure code** (sloppy code)
 - Reflects a flawed process or amateurish effort.
 - Provides easy opportunities for hackers.
- **Secure code**
 - Result of a deliberate process that prioritized security from the beginning.



Insecure vs. Secure Code /2

- Having a good secure code program and ways to verify and mitigate the creation of insecure code is paramount for any organization.
- System owner is responsible for code security.



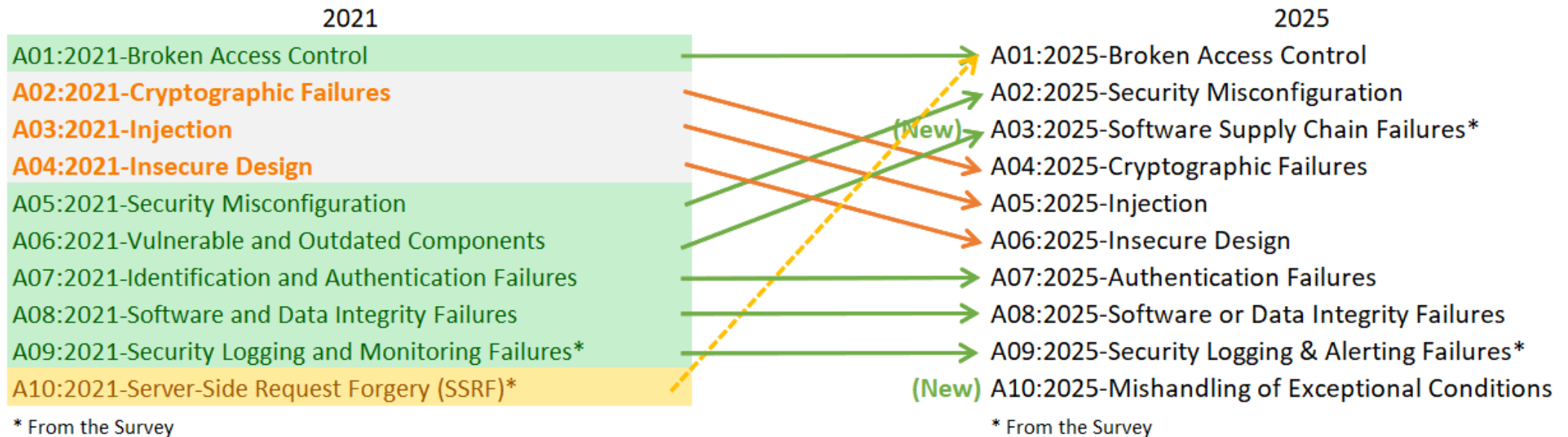
Development and Maintenance of More Secure Software

- **Question:** How can organizations encourage the development and maintenance of more secure software?
- One way is to adopt a systematic approach for software development.



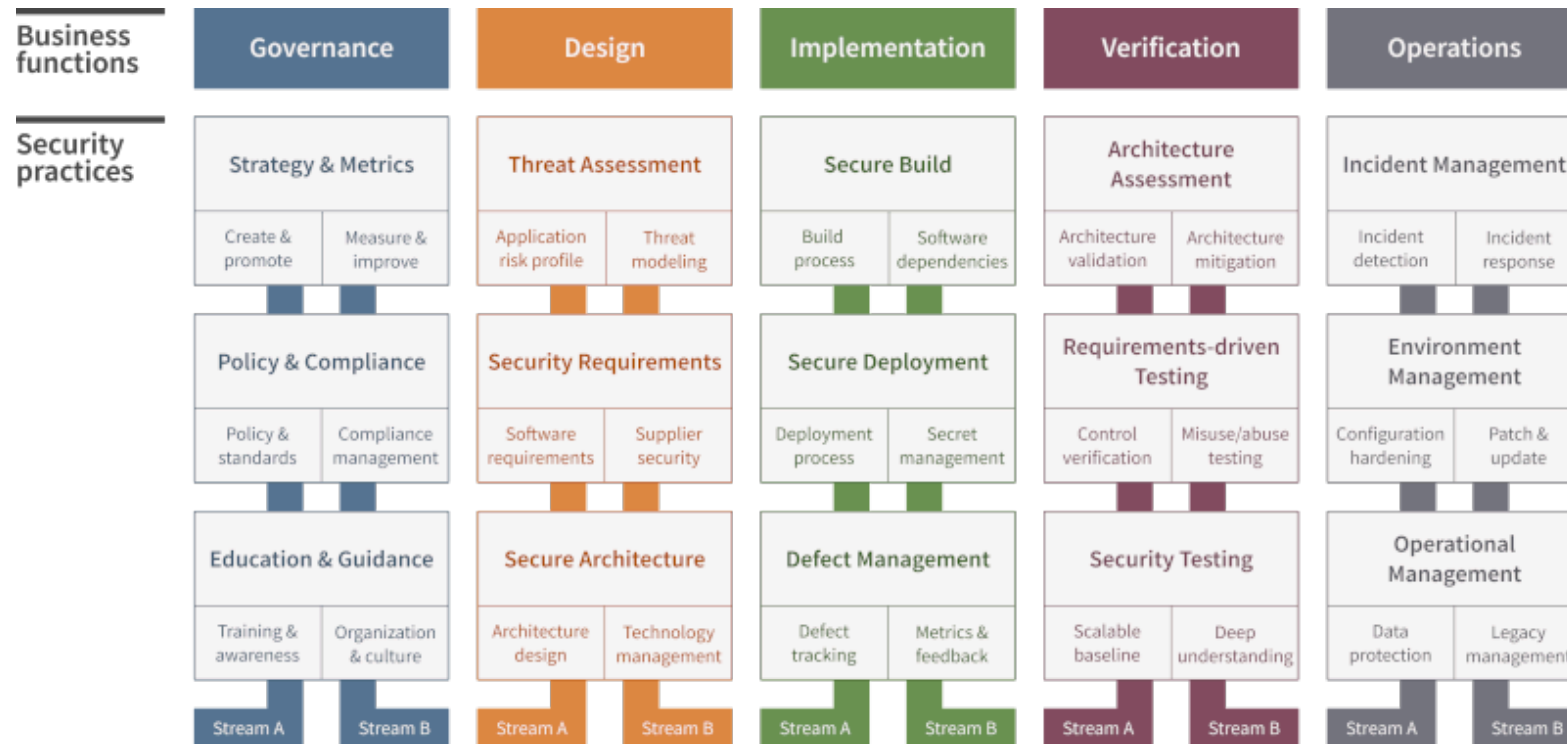
The Open Web Application Security Project (OWASP)

- Open community dedicated to enabling organizations to develop, purchase, and maintain applications that can be trusted.
- Every 3 years releases the top 10 most critical web application security flaws.



Software Assurance Maturity Model (SAMM)

- OWASP has created SAMM: <https://owaspsamm.org/>
- SAMM is an open framework to help organizations formulate and implement a strategy for software security that is tailored to the specific risks facing the organization.



Design - Security Requirements

Maturity Level	Security Requirements Objective	Stream A Software Requirements	Stream B Supplier Security
1	Consider security explicitly during the software requirements process.	High-level application security objectives are mapped to functional requirements.	Evaluate the supplier based on organizational security requirements.
2	Increase granularity of security requirements derived from business logic and known risks.	Structured security requirements are available and utilized by developer teams.	Build security into supplier agreements in order to ensure compliance with organizational requirements.
3	Mandate security requirements process for all software projects and third-party dependencies.	Build a requirements framework for product teams to utilize.	Ensure proper security coverage for external suppliers by providing clear objectives.

Design - Security Architecture

Maturity Level	Security Architecture Objective	Stream A Architecture Design	Stream B Technology Management
1	Insert consideration of proactive security guidance into the software design process.	Teams are trained on the use of basic security principles during design	Elicit technologies, frameworks and integrations within the overall solution to identify risk.
2	Direct the software design process toward known secure services and secure-by-default designs.	Establish common design patterns and security solutions for adoption.	Standardize technologies and frameworks to be used throughout the different applications
3	Formally control the software design process and validate utilization of secure components.	Reference architectures are utilized and continuously evaluated for adoption and appropriateness.	Impose the use of standard technologies on all software development.

Implementation – Secure Build

Maturity Level	Secure Build Objective	Stream A Build Process	Stream B Software Dependencies
1	Build process is repeatable and consistent.	Create a formal definition of the build process so that it becomes consistent and repeatable.	Create records with Bill of Materials of your applications and opportunistically analyze these.
2	Build process is optimized and fully integrated into the workflow.	Automate your build pipeline and secure the used tooling. Add security checks in the build pipeline.	Evaluate used dependencies and ensure timely reaction to situations posing risk to your applications.
3	Build process helps prevent known defects from entering the production environment.	Define mandatory security checks in the build process and ensure that building non-compliant artifacts fails.	Analyze used dependencies for security issues in a comparable way to your own code.

Implementation – Secure Deployment

Maturity Level	Secure Deployment Objective	Stream A Deployment Process	Stream B Secret Management
1	Deployment processes are fully documented.	Formalize the deployment process and secure the used tooling and processes.	Introduce basic protection measures to limit access to your production secrets.
2	Deployment processes include security verification milestones.	Automate the deployment process over all stages and introduce sensible security verification tests.	Inject secrets dynamically during deployment process from hardened storages and audit all human access to them.
3	Deployment process is fully automated and incorporates automated verification of all critical milestones.	Automatically verify integrity of all deployed software, independently on whether it's internally or externally developed.	Improve the lifecycle of application secrets by regularly generating them and by ensuring proper use.

Verification - Security Testing

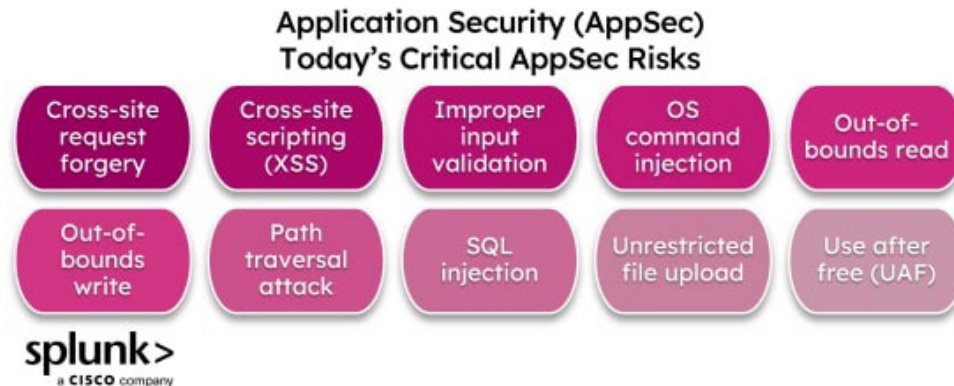
Maturity Level	Security Testing Objective	Stream A Scalable Baseline	Stream B Deep Understanding
1	Perform security testing (both manual and tool based) to discover security defects.	Utilize automated security testing tools	Perform manual security testing of high-risk components
2	Make security testing during development more complete and efficient through automation complemented with regular manual security penetration tests.	Employ application-specific security testing automation	Conduct manual penetration testing
3	Embed security testing as part of the development and deployment processes.	Integrate automated security testing into the build and deploy process	Integrate security testing into development process

Operations – Operational Management

Maturity Level	Security Testing Objective	Stream A Data Protection	Stream B Legacy Management
1	Foundational Practices	Implement basic data protection practices	Decommission unused applications and services as identified. Manage customer upgrades/migrations individually.
2	Managed, Responsive Processes	Develop data catalog and establish data protection policy.	Develop repeatable decommissioning processes for unused systems/services, and for migration from legacy dependencies. Manage legacy migration roadmaps for customers.
3	Active Monitoring and Response	Automate detection of policy non-compliance, and audit compliance periodically. Regularly review and update to data catalog and data protection policy.	Proactively manage migration roadmaps, for both unsupported end-of-life dependencies, and legacy versions of delivered software.

Code Security Issues

- The most common web application security flaw is the failure to properly validate input from the client or the environment.
- **Input Validation** is the process of validating all the input to an application before using it. This includes correct syntax, length, characters, and ranges.
 - Lack of input validation is potential for injection.
- **Output Encoding** is the process of making output safe before displaying (and in some cases, masking) the output of a process before it is provided to the recipient.
 - Example: displaying asterisks for numbers on a credit card receipt.

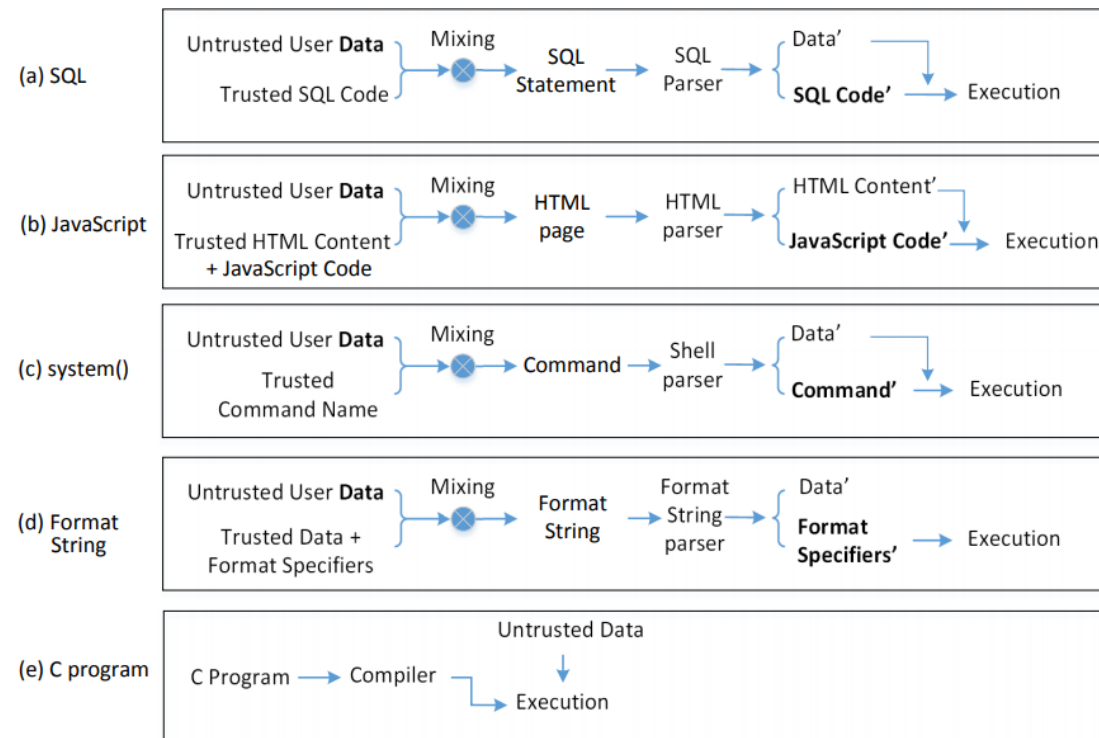


What Is Injection?

- Key Idea: Input data from the application is executed as code by the interpreter.
- Examples
 - SQL injection
 - Command injection
 - HTML script injection
 - Shell injection

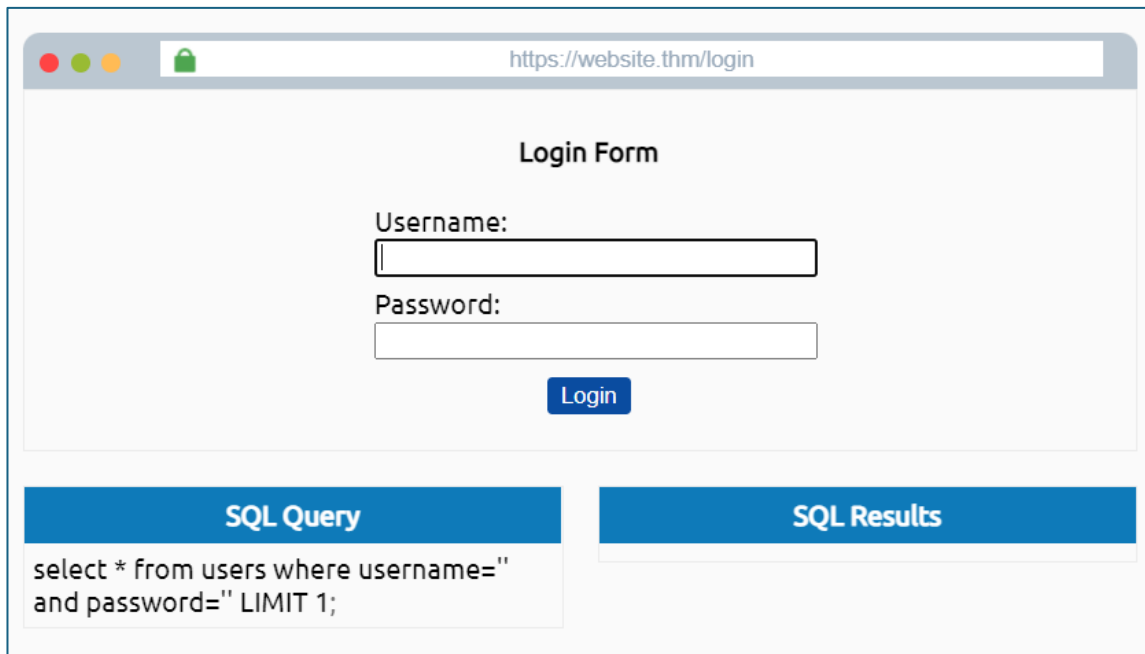
The Fundamental Cause

- **Mixing data and code** together is the cause of several types of vulnerabilities and attacks including SQL Injection attack, XSS attack, attacks on the system() function and format string attacks.

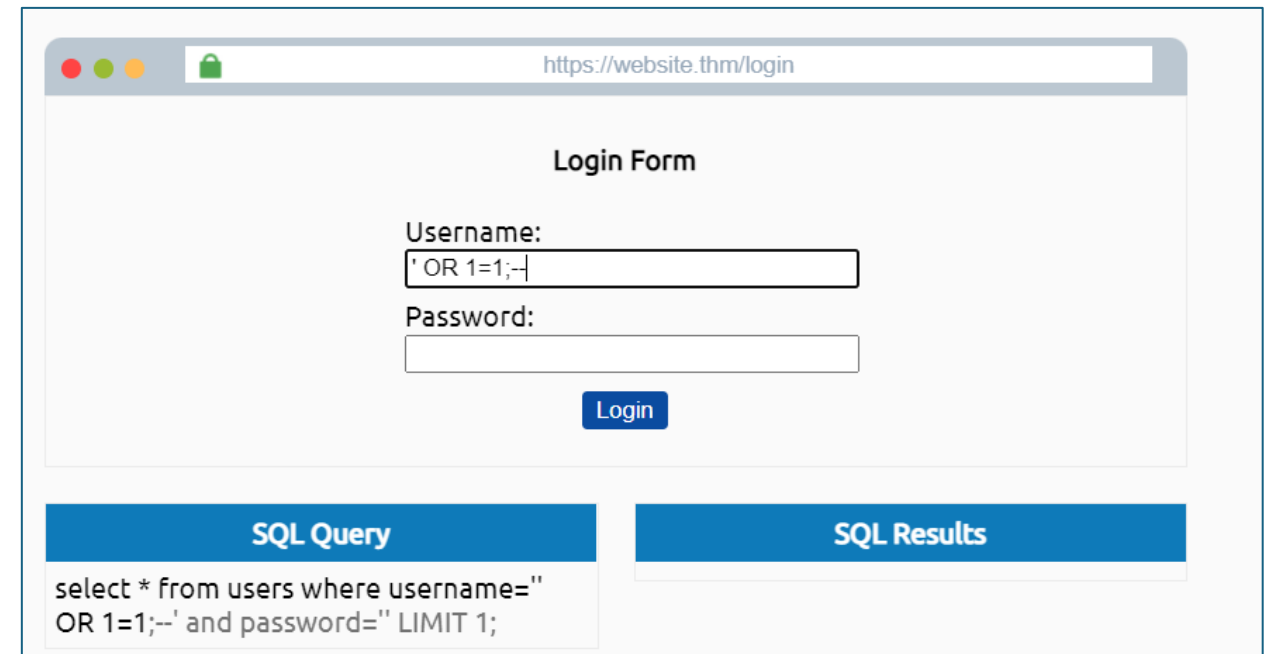


SQL Injection - Authentication Bypass Example

- `select * from users where username ='' and password='' LIMIT 1;`
- `select * from users where username ='' OR 1=1;--' and password='' LIMIT 1;`



The screenshot shows a web browser window with the address bar displaying `https://website.thm/login`. The page contains a "Login Form" with two input fields: "Username:" and "Password:". Below the form is a blue "Login" button. At the bottom of the page, there are two panels: "SQL Query" and "SQL Results". The "SQL Query" panel displays the query: `select * from users where username="" and password="" LIMIT 1;`. The "SQL Results" panel is empty.



The screenshot shows the same web browser window as the previous one, but with an SQL injection payload entered in the "Username:" field: `' OR 1=1;--'`. The "SQL Results" panel now displays the query: `select * from users where username="" OR 1=1;--' and password="" LIMIT 1;`.

In Practice

Application Development Policy

Synopsis: Define code and application development security requirements.

Policy Statement:

- System owners are responsible for oversight of secure code development.
- Security requirements must be defined and documented during the application development initiation phase.
- Code development will be done in accordance with industry best practices.
- Developers will be provided with adequate training, resources, and time.
- At the discretion of the system owner and with the approval of the Office of Information Security, third parties may be engaged to design, develop, and test internal applications.
- All code developed or customized must be tested and validated during development, prior to release, and whenever a change is implemented.
- The Office of Information Security is responsible for certifying the results of testing and accreditation to move to the next phase.

Topics

- Supply Chain Security
- System Security Requirements
- Secure Code
- Cryptography

Cryptography

- Three distinct goals are associated with cryptography:
 - **Confidentiality:** Unauthorized parties cannot access the data. Data can be encrypted to provide confidentiality.
 - **Integrity:** Assurance is provided that the data was not modified. Data can be hashed to provide integrity.
 - **Authenticity/nonrepudiation:** The source of the data is validated. Data can be digitally signed to ensure authentication/nonrepudiation and integrity.
- Data can be encrypted and digitally signed, which provides for confidentiality, authentication, and integrity.

Symmetric vs. Asymmetric Encryption

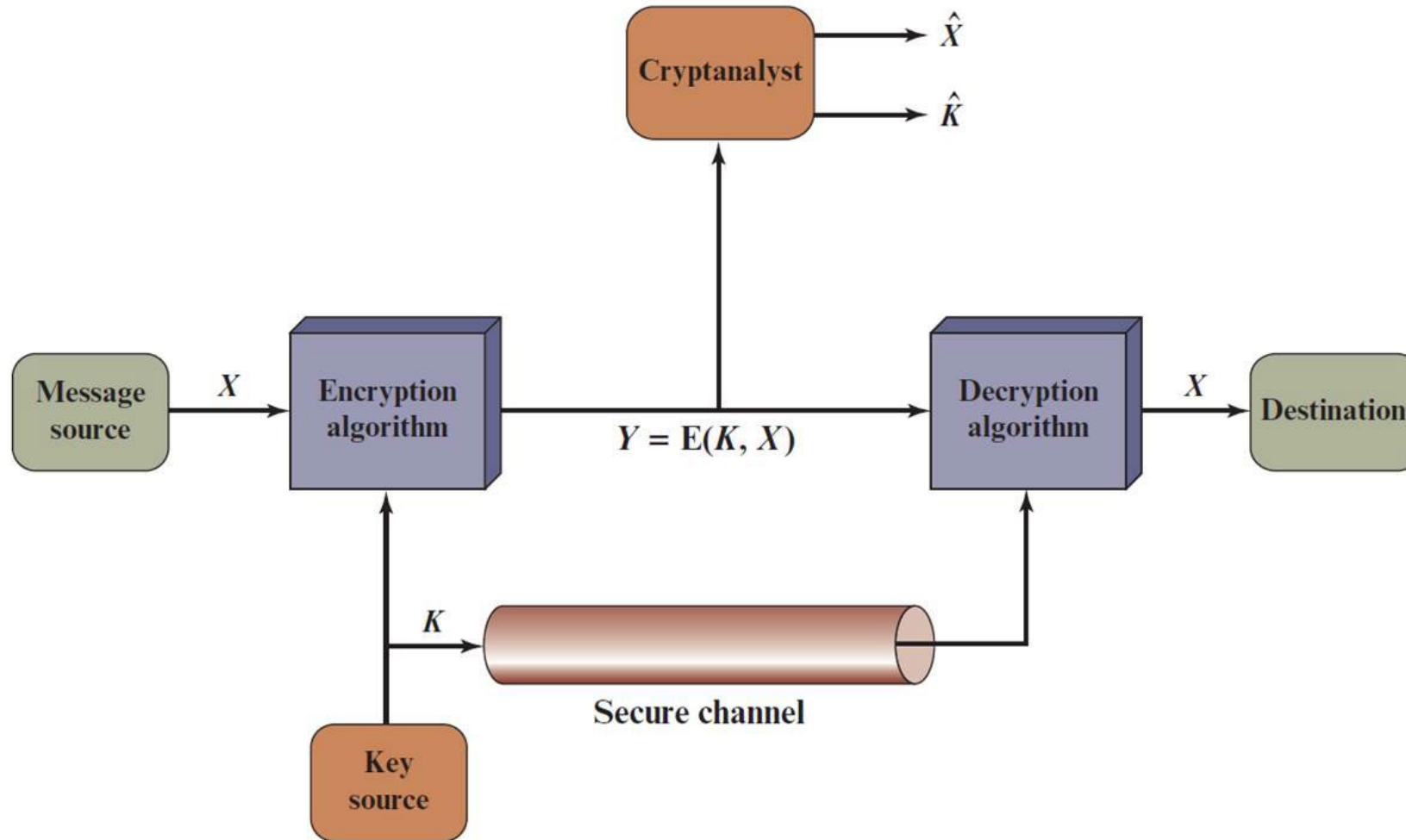
- **Symmetric Encryption**

- Uses a single secret key that must be shared in advance and kept private

- **Asymmetric Encryption**

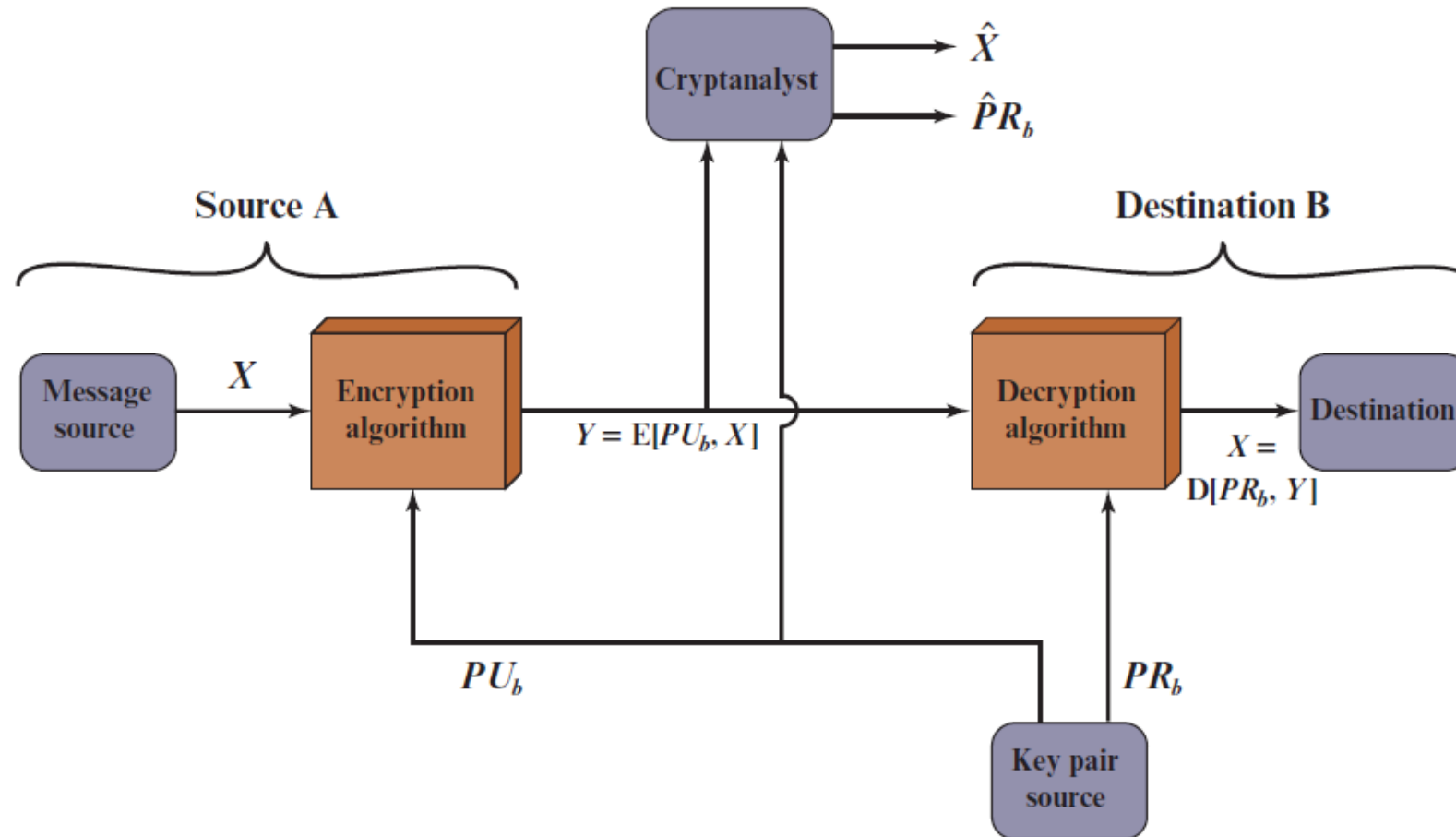
- Uses two different but mathematically related keys
- One is called public, and the other one is private

Symmetric Cryptosystem

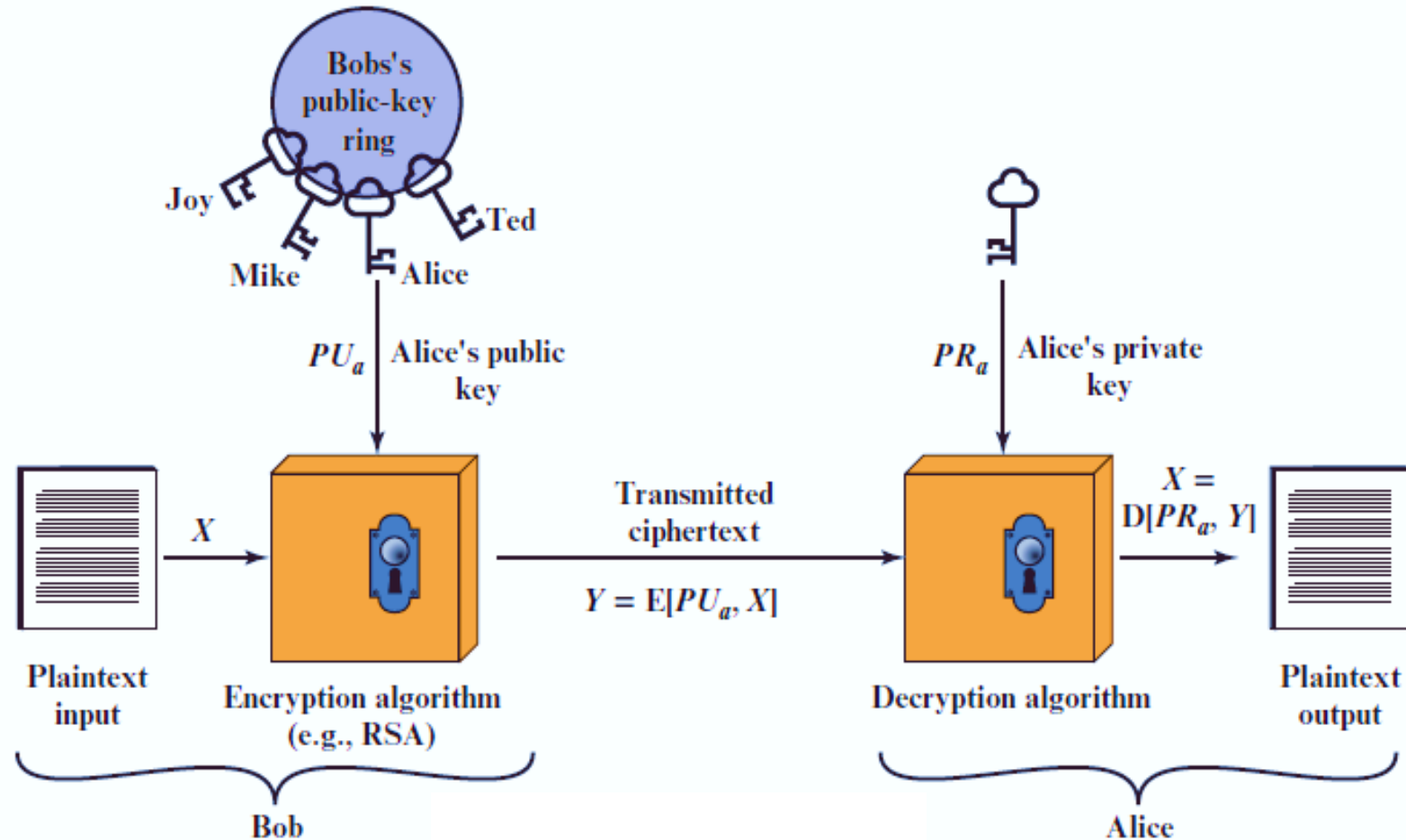


Source: Cryptography and Network Security, 8th Edition, by William Stallings

Public-Key Cryptosystem: Confidentiality

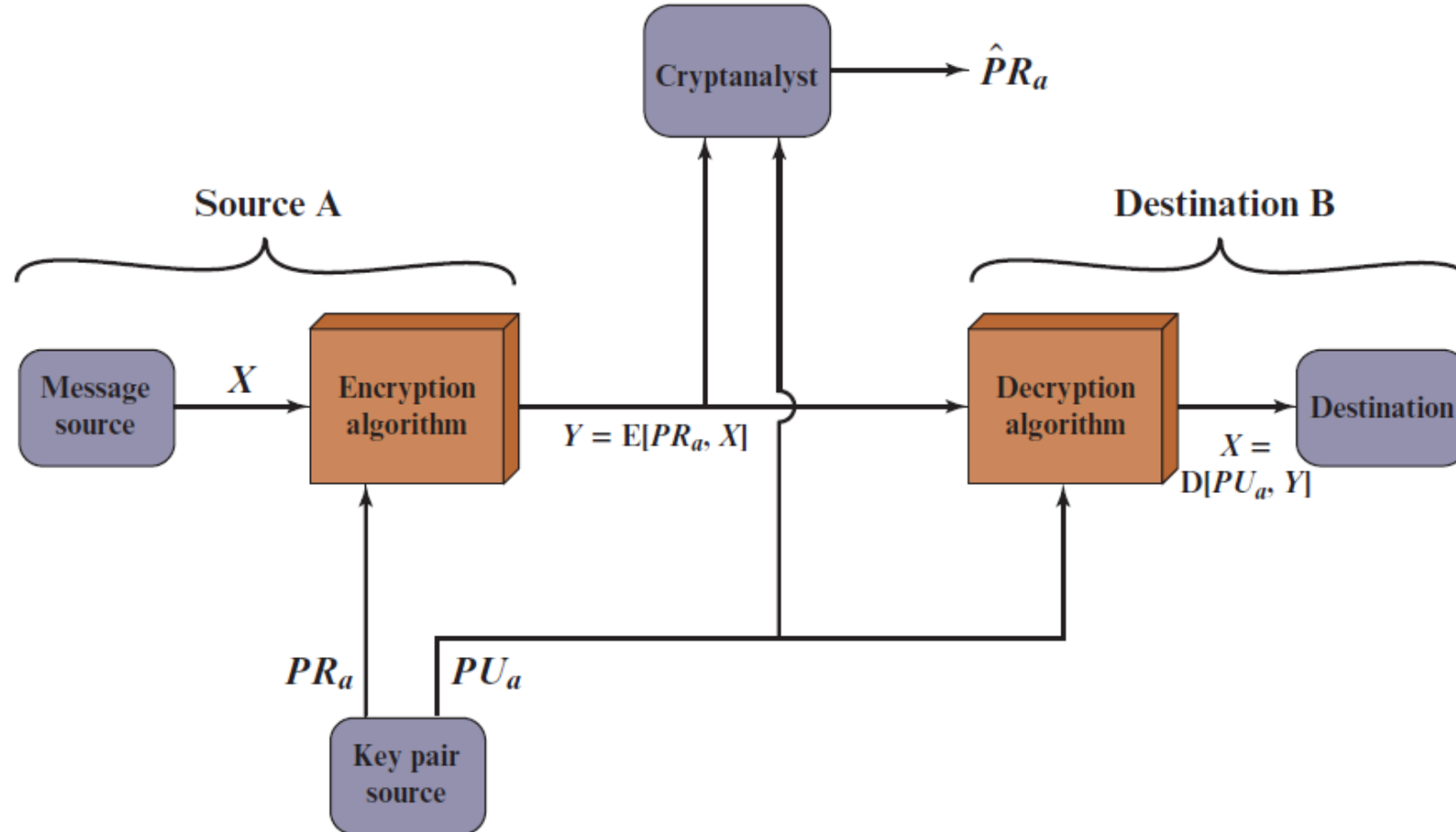


Encryption with Public Key – Confidentiality



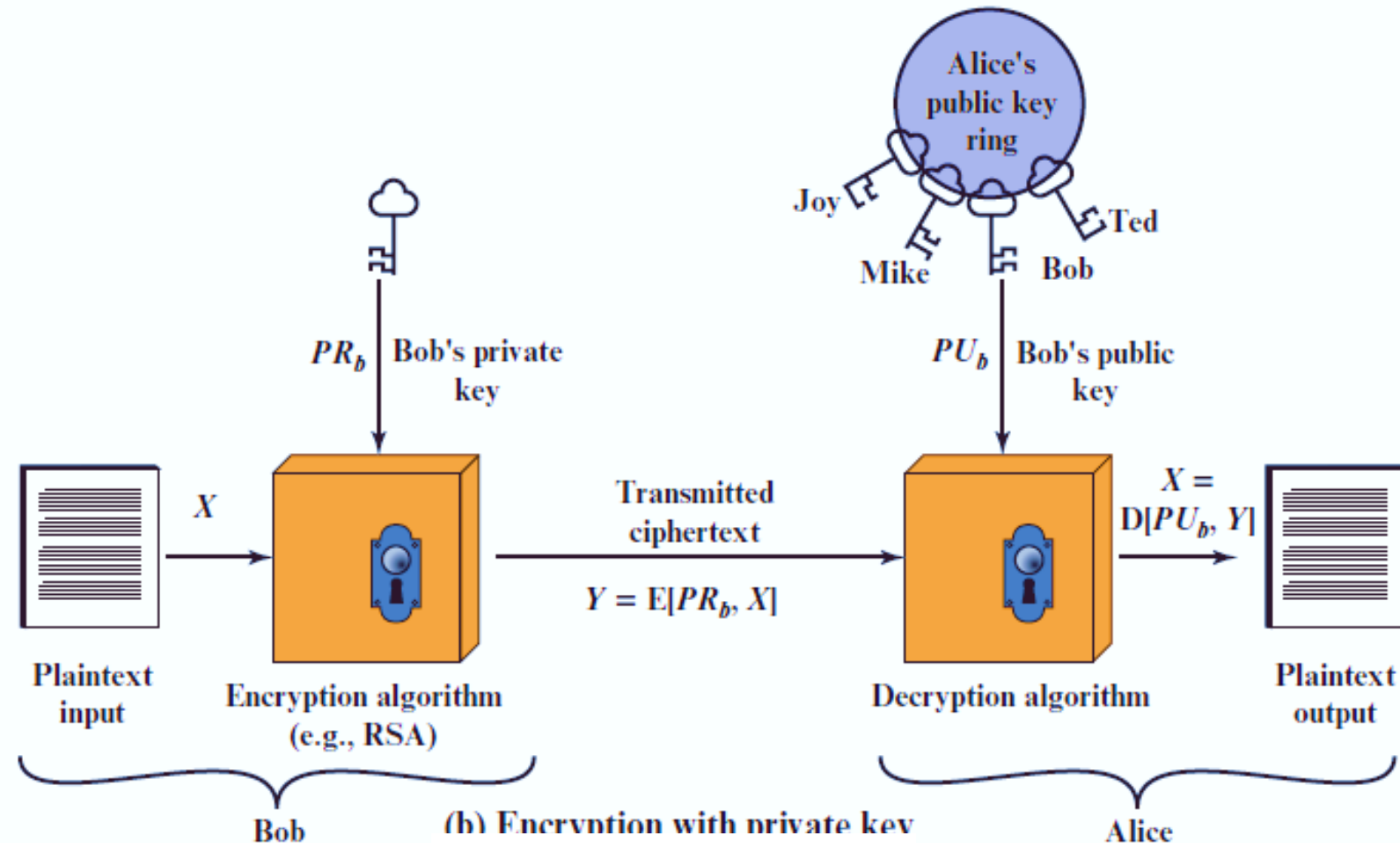
Source: Cryptography and Network Security, 8th Edition, by William Stallings

Public-Key Cryptosystem: Authentication



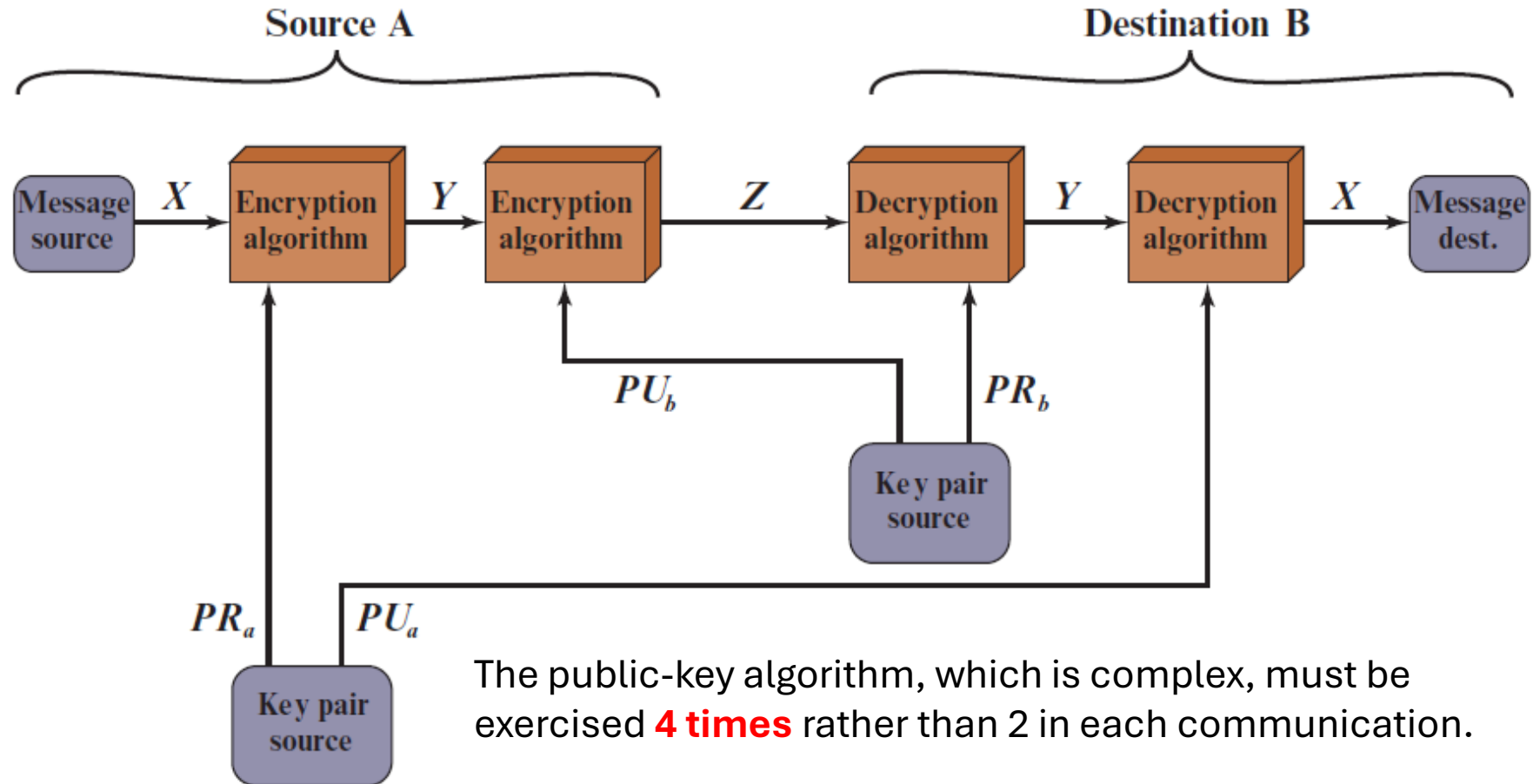
Source: Cryptography and Network Security, 8th Edition, by William Stallings

Encryption with Private Key – Authentication



Source: Cryptography and Network Security, 8th Edition, by William Stallings

Public-Key Cryptosystem: Authentication and Secrecy



Applications for Public-Key Cryptosystems

- Public-key cryptosystems can be classified into three categories:
 1. **Encryption/decryption:** The sender encrypts a message with the recipient's public key
 2. **Digital signature:** The sender “signs” a message with its private key, guarantees integrity and nonrepudiation.
 3. **Key exchange:** Two sides cooperate to exchange a session key
- Some algorithms are suitable for all three applications, whereas others can be used only for one or two

In Practice

Key Management Policy

Synopsis: To assign responsibility for key management and cryptographic standards.

Policy Statement:

- The Office of Information Security is responsible for key management, including but not limited to algorithm decisions, key length, key security and resiliency, requesting and maintaining digital certificates, and user education. The Office of Information Security will publish cryptographic standards.
- The Office of Information Technology is responsible for implementation and operational management of cryptographic technologies.
- Without exception, encryption is required whenever protected or confidential information is transmitted externally. This includes email and file transfer. The encryption mechanism must be NIST-approved.
- Without exception, all portable media that stores or has the potential to store protected or confidential information must be encrypted. The encryption mechanism must be NIST-approved.
- Data at rest must be encrypted regardless of media when required by state and/or federal regulation or contractual agreement.
- At all times, passwords and PINs must be stored and transmitted as cipher text.

Topics

- Supply Chain Security
- System Security Requirements
- Secure Code
- Cryptography

Summary

- Make security a priority during the design and acquisition phases of any new project
- Systems development lifecycle (SDLC) provides a standard process for any system development
- NIST's 5 phases of SDLC are Initiation, Development/Acquisition, Implementation, Operational, and Disposal
- Software release phases are Alpha, Beta, Release Candidate, and General Availability
- All companies should have a test environment. The more realistic it is, the more expensive, and the more accurate
- Secure code is the result of a deliberate process that prioritizes security from the beginning
- OWASP provides many tools and projects that can help
- Cryptography turns plain text into cipher text using a predetermined value called a key
- Hashing and digital signatures are two types of cryptography.
- Cryptographic keys can be symmetric or asymmetric.
- The components of PKI are CA, RA, client nodes, and digital certificate.